

Python Common Services Client

Table of Contents

Copyright Notice	1
Change History	1
1. Introduction	2
2. Installation	3
3. Common Services Client	4
3.1. Registration	4
3.2. Message Signing and Signature Verification	4
3.3. Sending Messages	5
3.4. Sending Requests	6
3.5. Message Subscription	8
3.6. Message Validation	8
4. Common Services Node	10
4.1. Sending Messages	10
4.2. Sending Requests	11
4.3. Message Subscription	11
4.3.1. <code>@subscriber</code> decorator	11
4.4. Capability and Status notifications	11
4.4.1. <code>@notify_capability</code> decorator	11
4.4.2. <code>@notify_status</code> decorator	12
5. File Transfer and Streaming Services	13
6. Capability and Status Services	14
6.1. Publishing	14
6.1.1. Security Marking	15
6.1.2. Node updates	15
6.2. Notifications	15
6.3. Subscribing	17
6.3.1. Direct store access	18
7. Configuration	20
7.1. JSON Configuration	20
7.1.1. CLI Options	20
8. OpenTelemetry Instrumentation	28
8.1. Metrics	28
8.1.1. Sending Messages	28
8.1.2. Received Messages	28
8.2. Traces	29
8.2.1. Sending Messages	29
8.2.2. Receiving Messages	29
Deprecated <code>request_response</code> function	29

8.2.3. Span Attributes	30
8.3. Logging	30
8.4. Client Applications	31

Copyright Notice

© 2019-2026 – CROWN OWNED COPYRIGHT. All Rights Reserved.

The copyright of this Software is vested in the Crown and the Software is the property of the Crown.

This Software is released in confidence, to the recipients nominated by the Crown and is for use only for the purposes of the Crown.

This Software shall not be copied or further disclosed except as authorised by the Crown. Any further disclosure shall be under the terms of this legend and any copies shall be the property of the Crown. On completion of the work in connection with which this Software is released the recipient shall return the Software (and any correspondence) to the issuing authority. <<<

Change History

1.0	Initial Version extracted from pycs-core v4.2
2.0	Update package name to pycs.client. Convert to async.
2.1	Add CommonServiceNode class
2.2	Add Designator class
2.3	Populate SILKWAVE security from MfDoc.
2.4	Validation improvements
2.5	Improve Cap/Sta filter handling. Add @client_application decorator.
2.6	Add config options for SSL connections
2.7	Separate application and silkwave security settings
2.8	Add --log-config option
3.0	BREAKING CHANGE: Transport abstraction
3.1	Add OpenTelemetry Support
3.2	Automatic cap/sta subscriptions

1. Introduction

The `pycs-client` package provides a `CommonServicesClient` which handles registration onto a Common Services network, and the ability to provide updates to the network regarding the Capability and Status of the client. It also allows for the client to subscribe to updates of capability and status from other clients.

2. Installation

pycs requires an installation of python at least version 3.13. The use of virtual environments is recommended.

```
$ python -m venv venv
$ ./venv/bin/pip install pycs.client
```

To use the pycs package interactively, run python inside the virtual environment

```
$ ./venv/bin/python
>>> import pycs.client
>>> help(pycs.client)
Help on package pycs.client:
...
```

3. Common Services Client

The `pycs.client` package provides a `CommonServicesClient` which handles registration onto a Common Services network, and the ability to provide updates to the network regarding the Capability and Status of the client. It also allows for the client to subscribe to updates of capability and status from other clients. The client object acts as an asynchronous context manager for use in an `async with` statement. It must be supplied with a configuration object and an active transport connection. Any transport conforming to the `pycs.transport.Transport` protocol may be used, for example a `pycs.silkwave.SilkwaveClient` instance. Multiple `CommonServicesClient` instances may connect using the same transport connection.

```
from pycs.client import CommonServicesClient, ClientConfig
from pycs.transport import Transport

async def example( transport: Transport):
    async with CommonServicesClient(
        config=ClientConfig(
            registrant_id='urn:csmf:pycs:example1',
            name='Example Client',
            registration_service='ns:cs:cshub.CS-Registration',
            kind='sensor',
        ),
        transport=transport,
    ) as client:
        ...
```

3.1. Registration

Once started, a `CommonServicesClient` will attempt to maintain a registration with the hub specified in its constructor. The context manager will not enter until an initial registration has been made. If the transport connection is lost, or the Common Services hub forces deregistration, any attempts to communicate with the network will suspend the coroutine and delay communications until the connection is restored and the client is re-registered with the hub.

3.2. Message Signing and Signature Verification

Common Services uses XML digital signatures to sign and verify messages. If a client is connecting to a Common Services hub that requires messages to be signed, then the `SigningConfig` of the initial client configuration must be populated before starting the client. The `certificate` property should contain a path to a PEM file containing the certificate or certificate and key. The key PEM can also be supplied separately using the `key` property. If the key is password protected, the password must be supplied using the `passphrase` property.

```
async def example( transport: Transport, certificate_pem: Path, passphrase: str):
    async with CommonServicesClient(config =
```

```

        ClientConfig(
            registrant_id='urn:csmf:pycs:example1',
            name='Example Client',
            registration_service='ns:cs:cshub.CS-Registration',
            kind='sensor',
            signing=SigningConfig(
                certificate=certificate_pem,
                passphrase=passphrase,
            ),
        ),
        transport=transport,
    ) as client:
        ...

```

By default the client will check signatures on all messages received, and will not allow unsigned messages or messages with an invalid or unrecognised signature to be processed. This behaviour can be controlled using the `check_signatures` and `allow_unsigned` properties on the `SigningConfig`. The initial `RegResponse` message from the hub should be signed, and client will validate that this message is signed by a trusted certificate. As the client uses OpenSSL under the hood, this trust will be established by querying the OpenSSL trust store (see OpenSSL documentation for further details), but it is also possible to supply the registration service certificate directly using the `registration_certificate` property.

If using the OpenSSL default trust, the client should be supplied with a list of valid X509 subjects that can be accepted, so as to avoid accepting a response that has been signed by a certificate within the trust chain, but not one that is the expected registration service. This can be done by supplying a list of valid X509 subjects to the `allowed_x509_subjects` property.

Once registration with the hub has completed, the client will automatically receive certificates for other clients on the network that are trusted by the hub, and validate all messages against these certificates.

3.3. Sending Messages

Messages may be sent to other Common Services clients by using the `send_message()` function. The `TokenMf`, `MsgDistMf` and `MsgMetadataMf` microformats on the message will be populated with appropriate values.

```

from pycs.client import *
from csmf.mf.common import *
from csmf.msg.rfsensor import TunerCommandRequest

async def example( client: CommonServicesClient, message: TunerCommandRequest):
    await client.send_message(
        message = message,
        destination = Registrant(
            registrantId='urn:csmf:pycs:sensor-1',

```



```
endPoint='net:example:hub:12345'))
```

3.4. Sending Requests

Common Services messages often come in request/response pairs, matched using the `requestId` field on the `MsgDistMf` microformat. If a single response is expected, then it may be awaited using the `send_request()` function. If a timeout is required, then wrapping in an `asyncio.timeout()` context is usually the most appropriate way of doing this.

In some situations, it may be more appropriate to request a callback using `request_callback`. The request object returned is a `Subscription`, allowing the request to be cancelled before the callback is called. The `Subscription` object acts as an `AsyncContextManager` for use in an `async with` statement, with automatic cancellation on context exit. The request is automatically cancelled on the first call to the callback, but the requestor may also cancel at any time.

If multiple responses are expected, then `request_responses` or `request_callbacks` may be used. `request_responses` returns an `AsyncIterator` which can be iterated over with an `async for` loop. `request_callbacks` is similar to `request_callback`, except that the subscription is not automatically cancelled on the first call to the callback.

```
import asyncio
from pycs.client import *
from pycs.csmf.mf import *
from pycs.csmf.msg.rfsensor import TunerCommandRequest, TunerCommandResponse

async def example(
    client: CommonServicesClient,
    message: TunerCommandRequest,
    destination: Registrant):

    async with client.send_request(
        message = message,
        destination = destination,
        expected_response=TunerCommandResponse,
    ) as response:
        ...

async def timeout_example(
    client: CommonServicesClient,
    message: TunerCommandRequest,
    destination: Registrant):
    try:
        async with (
            asyncio.timeout(10),
            client.send_request(
                message = message,
                destination = destination,
                expected_response=TunerCommandResponse,
```

```

        ) as response
    ):
        ...
except asyncio.TimeoutError:
    print("Timed Out")

async def single_callback_example(
    client: CommonServicesClient,
    message: TunerCommandRequest,
    destination: Registrant):
    async def handle_response(response: TunerCommandResponse):
        ...

    subs = await client.request_callback(
        message = message,
        destination = destination,
        expected_response=TunerCommandResponse,
        callback = handle_response)
    ...

async def iterator_example(
    client: CommonServicesClient,
    message: TunerCommandRequest,
    destination: Registrant)
    async for response in client.request_responses(
        message = message,
        destination = destination,
        expected_response=TunerCommandResponse):
        ...

async def multiple_callbacks_example(
    client: CommonServicesClient,
    message: TunerCommandRequest,
    destination: Registrant):
    async def handle_response(response: TunerCommandResponse):
        ...

    subscription = await client.request_callbacks(
        message = message,
        destination = destination,
        expected_response=TunerCommandResponse,
        callback = handle_response)
    ...
    subscription.cancel()

async def multiple_callbacks_context_example(
    client: CommonServicesClient,
    message: TunerCommandRequest,
    destination: Registrant)

```

```

async def handle_response(response: TunerCommandResponse):
    ...

async with await client.request_callbacks(
    message = message,
    destination = destination,
    expected_response=TunerCommandResponse,
    callback = handle_response)
    ...

```

3.5. Message Subscription

To receive messages from other Common Services clients, a client can use the `subscribe` or `subscribe_callback` functions. Similarly to request/response, there are two modes of operation for subscriptions. `subscribe` returns an `AsyncIterator` allowing an `async for` loop over the responses, whereas `subscribe_callback` ensures that the supplied callback is called for each message of the correct type received.

```

from pycs.client import *
from pycs.csmf.msg.rfSENSOR import TunerCommandRequest

async def example(client: CommonServicesClient):
    async for request in client.subscribe(TunerCommandRequest, span_name="Process Tuner Request"):
        ...

async def handle_request(request: TunerCommandRequest):
    ...

async def callback_example(client: CommonServicesClient):
    subscription = await client.subscribe_callback(TunerCommandRequest, handle_request, span_name="Process Tuner Request")
    ...
    subscription.cancel()

```

See also [Section 4.3, “Message Subscription”](#) for a simpler method of subscribing when using `CommonServicesNode`.

3.6. Message Validation

By default the `CommonServicesClient` will validate all incoming and outgoing messages against the message definitions, and warn if an invalid message is sent or received. For Capability and Status service updates, the current full document is validated after applying any abbreviated updates.

The validation may be turned on and off using the `incoming` and `outgoing` properties on the `ValidationConfig` of the constructor configuration, or the corresponding `validation` property on the instance. This may prove useful if, for example, you are writing a client to test the effect of invalid messages on the network.

```

async def example(transport: Transport):

```

```

async with CommonsServicesClient(config=ClientConfig(
    registrant_id='urn:csmf:pycs:example1',
    name='Example Client',
    registration_service='ns:cs:cshub.CS-Registration',
    kind='sensor',
    validation=ValidationConfig(
        incoming=False,
        outgoing=True,
    )
),
    transport=transport,
) as client:
    ...
    client.validation.incoming = True

```

4. Common Services Node

The `pys.client` package provides a `CommonServicesNode` class, intended for users to subtype. This makes it easier to group capability and status and send receive messages from a single Common Services node. The node object acts as an asynchronous context manager for use in an `async with` statement. Alternatively the `start()` and `close()` functions may be called explicitly. When the node is closed, any outstanding `subscriptions` are cancelled.

The node also provides a `stop()` function, and a `finished()` awaitable, which can be used to prevent and `async with` exiting immediately in cases when the node simply wants to sit and process incoming messages.

```
from pys.client import *
import asyncio

async def stop_sensor(sensor):
    await asyncio.sleep(60)
    await sensor.stop()

async def run_context(client: CommonServicesClient):
    async with CommonServicesNode(client = client, node_id='sensor-1') as sensor:
        ...
        stopper = asyncio.create_task(stop_sensor(sensor))
        ...
        await sensor.finished()
        await stopper

async def run_explicit(client: CommonServicesClient):

    sensor = CommonServicesNode(client=client, node_id='sensor-1')
    await sensor.start()
    try:
        ...
        await sensor.finished()
    finally:
        await sensor.close()
```

4.1. Sending Messages

Messages may be sent to other Common Services clients by using the `send_message()` function. This simply delegates to the parent client, but sets the `node_id` parameter appropriately, which in turn sets the `MsgDistMf.originator.nodeId` on the outgoing message.

```
from pys.client import *
from csmf.mf.common import *
from csmf.msg.rfsensor import TunerCommandRequest
```

```

async def example( node: CommonServicesNode, message: TunerCommandRequest, destination: Registrant):
    await node.send_message(
        message = message,
        destination = destination)

```

4.2. Sending Requests

The functions `request`, `request_responses`, `request_callback` and `request_callbacks` are available, which delegate to similar functions on the parent client, setting the `node_id` parameter as appropriate. Any outstanding requests are cancelled when the node is closed.

4.3. Message Subscription

To receive messages from other Common Services clients, a node can use the `subscribe` or `subscribe_callback` functions. These delegate through to similar functions on the parent client, setting the `node_id` parameter as appropriate. Only messages of the correct type and directed to this `node_id` will be received. Any active subscriptions are cancelled when the node is closed.

4.3.1. `@subscriber` decorator

To simplify the act of subscribing to messages, a function may be decorated with `@subscriber`. This will automatically subscribe to the supplied message when the node starts, with the annotated function as the callback.

```

from pycs.client import *
from csmf.msg.rfsensor import TunerCommandRequest

class ExampleSensor(CommonServicesNode):

    @subscriber(TunerCommandRequest)
    async def handle_tune_request(self, request: TunerCommandRequest):
        ...

```

4.4. Capability and Status notifications

4.4.1. `@notify_capability` decorator

To simplify receiving notifications of Capability messages, a function may be decorated with `@notify_subscriber`, passing the capability class to be notified of as a parameter. This will register the decorated function to receive any Capability notifications by automatically calling `capability_service.add_notification`.

This will automatically subscribe to appropriate updates from the capability service, unless the `subscribe` parameter is set to `False`. See [Section 6.3, “Subscribing”](#) for further details.

```
from pycs.client import *
from csmf.msg.rfsensor import Capability

class ExampleSensor(CommonServicesNode):

    @notify_capability(Capability)
    async def handle_cap_update(self, reg_id, node_id, doc_type, doc) :
        if doc:
            print(doc.to_yaml())
        else:
            print('Document removed')
```

4.4.2. @notify_status decorator

The `@notify_status` decorator works exactly as `@notify_capability`, but for the Status service.

5. File Transfer and Streaming Services

The file transfer and streaming service of the underlying `Transport` are exposed as `file_transfer_service` and `stream_service` properties on the `CommonServicesClient`, and are used as specified in ``pys.transport`` documentation.

6. Capability and Status Services

The Common Services Client exposes Capability and Status services through `capability_service` and `status_service` properties on the `CommonServicesClient`. Both of these work in an identical manner, so below we will concentrate solely on the Capability Service, but all information is equally applicable to the Status Service.

6.1. Publishing

A capability may be published to the CS Network by calling `client.capability_service.update()`. A local store of the current state is maintained, and on each call to `update`, a minimal set of changes will be sent to the network Capability Service. Where possible an Abbreviated Update will be used, but the full state is maintained, and if the network requests a full update this will be sent and used as the basis for subsequent abbreviated updates.

It is recommended that an application maintains a local Capability document as defined in one of the generated Message classes, and simply calls the update method whenever there is a change to this document that needs publicising to the wider network.

Any number of different Capability documents may be maintained, and each will be sent to the network separately as it is updated.

```
from pycs.client import *
import csmf.msg.rfsensor

class Sensor:
    def __init__(self, client: CommonServicesClient)
        self.client = client
        self.capability = csmf.msg.rfsensor.Capability()
        self.platform = self.capability.PlatformCapabilityMf
        self.platform.platformType = 'groundFixed'
        self.platform.DesignatorMf.designator.append(...)
        self.platform.DesignatorMf.designator[-1].designatorType = 'displayName'
        self.platform.DesignatorMf.designator[-1].name = 'Example Platform 1'
        self.sensor = self.capability.SensorCapabilityMf
        self.sensor.nodeId = 'example:sensor:1'
        self.sensor.DesignatorMf.designator.append(...)
        self.sensor.DesignatorMf.designator[-1].designatorType = 'displayName'
        self.sensor.DesignatorMf.designator[-1].name = 'Example Sensor 1'
        self.tuner = self.sensor.TunerCapabilityMf.append(...)
        self.tuner.maxIfBandwidth = ...
        self.tuner.maxIfBandwidth.whole = 400000000
        self.tuner.supportsDirectTasking = False
        self.tuner.supportsTunerCommanding = False
        self.tuner.RfCoverageBandMf = ...
        self.tuner.RfCoverageBandMf.frequencyRange = ...
        self.tuner.RfCoverageBandMf.frequencyRange.Min = ...
        self.tuner.RfCoverageBandMf.frequencyRange.Min.whole = 1000000000
        self.tuner.RfCoverageBandMf.frequencyRange.Max = ...
```

```

        self.tuner.RfCoverageBandMf.frequencyRange.Max.whole = 800000000

    async def __aenter__(self):
        await self.start()
        return self

    async def __aexit__(self, exc_type, exc_val, exc_tb):
        pass

    async def start(self):
        await self.client.registered()
        await self.update_capability()

    async def set_range(self, min:int, max:int):
        self.tuner.RfCoverageBandMf.frequencyRange.Min.whole = min
        self.tuner.RfCoverageBandMf.frequencyRange.Max.whole = max
        await self.update_capability()

    async def update_capability(self):
        self.client.capability_service.update(self.capability)

    async def example(client: CommonServicesClient):
        async with Sensor(client) as sensor:
            await sensor.set_range(200000000,400000000)

```

6.1.1. Security Marking

The security marking for the update message may be set by assigning an appropriate value to `client.capability_service.security`. This should be set to the maximum classification of any of the individual capability documents withing the local store. It will default to the base classification of the client at the time of creation.

```

self.client.capability_service.security.classification = 'S'
self.client.capability_service.security.relTo = 'UK'

```

6.1.2. Node updates

When using the `CommonServicesNode` class, capability and status updates may be more easily published using the `update_capability()` and `update_status()` member functions.

6.2. Notifications

A client may be notified of changes to remote clients by supplying a callback to the `add_notification` function on the service, or with the async generator `notify`. The `reg_id`, `node_id` and `doc_type` parameters may be used to restrict which updates will cause a notification. If `node_id` is supplied, then `reg_id` must be supplied too. The `doc_type` may either be a message class, or an id string. Each

time the document is updated, the callback will be called passing the registrant id, node id and document type, along with the latest version of the document. If a document is removed by the CS network, then the callback will be called with the document set to None. The `add_notification` function returns a `Subscription` instance which can be used as a context manager, or explicitly cancelled once the caller is no longer interested in notifications.

```
from pycs.client import *
import csmf.msg.rfsensor
import asyncio

def log_update(update: UpdateNotification):
    if update.doc:
        print(update.doc.to_yaml())
    else:
        print('Document removed')

async def all_updates(client: CommonServicesClient):
    async for update: UpdateNotification in client.capability_service.notify():
        log_update(update)

async def rfreg_updates(client: CommonServicesClient):
    async for update: UpdateNotification in client.capability_service.notify(
        reg_id='registrant-1',
        doc_type=csmf.msg.rfsensor.Capability):
        log_update(update)

async def node_updates(client: CommonServicesClient):
    async for update: UpdateNotification in client.capability_service.notify(
        reg_id='registrant-1',
        node_id='node-1',
        doc_type=csmf.msg.rfsensor.Capability):
        log_update(update)

async def example(client: CommonServicesClient):
    asyncio.create_task(all_updates)
    asyncio.create_task(rfreg_updates)
    asyncio.create_task(node_updates)
```

Notification callbacks

```
from pycs.client import *
import csmf.msg.rfsensor
import asyncio

async def log_update(reg_id, node_id, doc_type, doc):
    if doc:
        print(doc.to_yaml())
    else:
        print('Document removed')
```

```

async def example(client: CommonServicesClient):
    all_subscription = client.capability_service.add_notification(log_update)
    rf_reg_subscription = client.capability_service.add_notification(
        callback=log_update,
        reg_id='registrant-1',
        doc_type=csmf.msg.rfsensor.Capability)

    node_subscription = client.capability_service.add_notification(
        callback=log_update,
        reg_id='registrant-1',
        node_id='node-1',
        doc_type='urn:csmf:rfsensor:capability')

```

See also [Section 4.4, “Capability and Status notifications”](#) for a neater way to do this when using the `CommonServicesNode` class.

6.3. Subscribing

The notification functions above will automatically subscribe using a filter that returns appropriate documents. If a more restrictive filter is required, then the `subscribe` parameter can be set to `False`, and an explicit `subscribe` call made with an appropriate filter.

This function can also be useful if the application needs to maintain a view of the network capability or status, but does not require notification of changes.

This will subscribe to updates from the Common Services network that match the given filter, or to all updates if no filter is supplied. The filter will be combined with any active automatic notification filters. Note that this may have the effect of negating any extra restrictions if the automatic filters are less restrictive.

The filter may be specified as a tree of criteria from the `pycs.client.filter` package, but can more easily be parsed from a string using the filter DSL as defined in the BNF below.

```

filter      := expression
expression  := or_exp
or_exp      := and_exp ( '|' and_exp )*
and_exp     := not_exp ( '&' not_exp )*
not_exp     := [ '!' not_exp | parenthesised ]
parenthesised := [ '(' expression ')' | criteria ]
criteria    := microformat '(' definition ( '&' definition ) ')'
microformat  := NAME
definition   := field [ '=' | '!=' | '<' | '<=' | '>' | '>=' ] value
field       := NAME
value       := [ BOOLEAN | ''' STRING ''' | INTEGER | REAL | TIMESTAMP | DURATION | 'uri[' STRING ']' | 'token[' STRING ']' ]

```

```

from pycs.client import CommonServicesClient, Filter
from pycs.rfexgeo import Capability

async def example(client : CommonServicesClient):
    await client.capability_service.subscribe(Filter.parse('PlatformCapabilityMf(platformType = "groundFixed)'))

```

Alternatively, the filter string can be passed in directly.

```
await client.capability_service.subscribe('PlatformCapabilityMf(platformType = "groundFixed")')
```

A very common scenario for a filter is requesting messages with a certain message type. This could be set up using a filter as follows

```
await client.capability_service.subscribe('MsgDistMf(type/id="urn:csmf:rfextgeo:Capability")')
```

Because this scenario is so common, a shortcut to the above is to supply the Capability class type as the argument, or just the type id string.

```
await client.capability_service.subscribe(Capability)
```

The subscribe function can take multiple different filters as parameters. In this case the messages will be received if any of the filters match.

```
await client.capability_service.subscribe(Capability, 'PlatformCapabilityMf(platformType = "groundFixed")')
```

which is equivalent to

```
await client.capability_service.subscribe(Capability,
    'MsgDistMf(type/id="urn:csmf:rfextgeo:Capability") | PlatformCapabilityMf(platformType = "groundFixed")')
```

Calling `unsubscribe()` will remove any active subscriptions. A subscription may be updated by calling `subscribe()` again, but note that the Common Services documentation recommends that this is not done more often than every 10 minutes due to the network traffic that might be generated.

```
from pycs.client import *
import asyncio

async def example(client: CommonServicesClient):
    await client.capability_service.subscribe(Filter.parse('PlatformCapabilityMf(platformType = "groundFixed")'))

    await asyncio.sleep(10*60)

    await client.capability_service.unsubscribe()
    await client.capability_service.subscribe(Filter.parse('PlatformCapabilityMf(platformType = "groundMobile")'))
```

6.3.1. Direct store access

The `remote_store` attribute on the service contains a multi-level dictionary, firstly keyed on registrant id, then by node id, and then by document type id. If the node id of the original document was `None` then the node id will be equal to the registrant id. The document with this selector may then be accessed using the `doc` attribute of the returned object.

Additionally, a collection of all documents of a particular type may be obtained from the `docs` function on the service by passing either a message class or a message uri.

```
from pycs.client import *
import csmf.msg.rfsensor

async def example(client: CommonServicesClient):
    doc = client.capability_service.remote_store['registrant-1']['node-1']['urn:csmf:rfsensor:capability'].doc
    docs = client.capability_service.docs(csmf.msg.rfsensor.Capability)
    docs = client.capability_service.docs('urn:csmf:rfsensor:capability')
```

7. Configuration

7.1. JSON Configuration

All of the configuration classes are Pydantic models, and so can be populated from a JSON configuration file. See the [Pydantic documentation](#) for more details.

```
from pycs.client import CommonServicesClient, ClientConfig
from pycs.transport import Transport
from pathlib import Path

async def example(transport: Transport):
    config = ClientConfig.model_validate_json(Path('config.json').read_text())
    async with CommonServicesClient(config=config, transport=transport) as sw:
        ...
```

7.1.1. CLI Options

An `ApplicationSettings` class is provided to assist with supplying config data as a command line parameters. This class utilises [Pydantic Settings](#) to read configurations from the command line, environment variables and other sources.

This class should be used as the base class of a configuration object with your application options. It provides additional options to allow supplying a JSON config file, printing the schema for the config file, and specifying a [log configuration file](#).

Default values may be overridden by partially populating or subclassing the default value config classes. In the example below the `kind` property is suppressed from the CLI and forced to have the value `sensor` by subclassing `ClientConfig`. The `registration_service` property is also suppressed from the CLI and a value is calculated from the hub name and SILKWAVE transport configuration.

```
import asyncio
from typing import Literal

from pydantic_settings import CliSuppress

from pycs.client import CommonServicesClient, ClientConfig, ApplicationSettings, OTELConfig
from pycs.silkwave import SilkwaveClient, SilkwaveConfig

class CSSilkwaveConfig(SilkwaveConfig):
    reg_service: CliSuppress[str] = "CS-Registration"
    cs_ns: CliSuppress[str] = "ns:cs"

    def registration_service(self, hub):
        return f"{self.cs_ns}:{hub}.{self.reg_service}"
```

```

class SensorConfig(ClientConfig):
    kind: CliSuppress[Literal["sensor"]] = "sensor"
    registration_service: CliSuppress[str | None] = None

class ExampleSettings(ApplicationSettings, cli_avoid_json=True):
    silkwave: CSSilkwaveConfig = CSSilkwaveConfig()
    client: SensorConfig = SensorConfig()
    otel: OTELConfig = OTELConfig()
    cs_hub: str = "cshub"

    def model_post_init(self, context, /):
        self.client.registration_service = (
            self.client.registration_service
            or self.silkwave.registration_service(self.cs_hub)
        )
        super().model_post_init(context)

async def run():
    config = ExampleSettings()

    config.otel.configure()

    async with SilkwaveClient(config=config.silkwave) as transport:
        async with CommonServicesClient(config=config.client, transport=transport):
            ...

if __name__ == "__main__":
    try:
        asyncio.run(run())
    except KeyboardInterrupt:
        pass

```

```

> python -m pycs.silkwave.example_client --help
usage: python -m pycs.client.example_client [-h] [--config Path]
                                           [--config-schema | --no-config-schema]
                                           [--log-config Path]
                                           [--silkwave.name str]
                                           [--silkwave.alias.nameserver str]
                                           [--silkwave.alias.name str]
                                           [--silkwave.alias.prefix-hub bool]
                                           [--silkwave.security.classification {U,R,C,S,TS}]
                                           [--silkwave.security.sci-controls str]
                                           [--silkwave.security.releaseable-to str]
                                           [--silkwave.security.access-controls str]
                                           [--silkwave.security.owner-producer str]
                                           [--silkwave.security.creator-organization str]
                                           [--silkwave.security.classified-by str]
                                           [--silkwave.security.classification-reason str]
                                           [--silkwave.security.derived-from str]
                                           [--silkwave.security.declass-date str]
                                           [--silkwave.security.declass-exception str]
                                           [--silkwave.security.auth-id str]
                                           [--silkwave.security.alt-label str]

```



```

[--silkwave.connection.host str]
[--silkwave.connection.port int]
[--silkwave.connection.username str]
[--silkwave.connection.password str]
[--silkwave.connection.ssl.key Path]
[--silkwave.connection.ssl.passphrase str]
[--silkwave.connection.ssl.certificate Path]
[--silkwave.connection.ssl.ca Path]
[--silkwave.connection.ssl.ca-dir Path]
[--silkwave.files.force-hub-hostname str]
[--silkwave.files.force-hub-port int]
[--silkwave.files.https.key Path]
[--silkwave.files.https.passphrase str]
[--silkwave.files.https.certificate Path]
[--silkwave.files.https.ca Path]
[--silkwave.files.https.ca-dir Path]
[--silkwave.streams.force-hub-hostname str]
[--silkwave.streams.listener-default.host str]
[--silkwave.streams.listener-default.bind str]
[--silkwave.streams.listeners list[JSON]]
[--client.registrant-id str]
[--client.security.classification {U,R,C,S,TS}]
[--client.security.sci-controls str]
[--client.security.releaseable-to str]
[--client.security.access-controls str]
[--client.security.owner-producer str]
[--client.security.creator-organization str]
[--client.security.classified-by str]
[--client.security.classification-reason str]
[--client.security.derived-from str]
[--client.security.declass-date str]
[--client.security.declass-exception str]
[--client.security.auth-id str]
[--client.security.alt-label str]
[--client.signing.key Path]
[--client.signing.passphrase str]
[--client.signing.certificate Path]
[--client.signing.ca Path]
[--client.signing.allowed-x509-subjects list[str]]
[--client.signing.registration-certificate Path]
[--client.signing.check-signatures | --no-client.signing.check-signatures]
[--client.signing.allow-unsigned | --no-client.signing.allow-unsigned]
[--client.validation.incoming bool]
[--client.validation.outgoing bool]
[--client.force-span-context bool]
[--otel.service.name str]
[--otel.service.namespace str]
[--otel.connection.host str]
[--otel.connection.port int]
[--otel.connection.ssl.key Path]
[--otel.connection.ssl.passphrase str]
[--otel.connection.ssl.certificate Path]
[--otel.connection.ssl.ca Path]
[--otel.connection.ssl.ca-dir Path]
[--otel.tracer.max-queue-size int]
[--otel.tracer.max-export-batch-size int]
[--otel.tracer.schedule-delay-millis float]
[--otel.tracer.export-timeout-millis float]
[--otel.metrics.export-interval-millis float]
[--otel.metrics.export-timeout-millis float]
[--otel.log.max-queue-size int]
[--otel.log.max-export-batch-size int]
[--otel.log.schedule-delay-millis float]
[--otel.log.export-timeout-millis float]
[--otel.log.level {CRITICAL,ERROR,WARNING,INFO,DEBUG}]
[--cs-hub str]

```

options:

-h, --help show this help message and exit
--config Path Load configuration from YAML/JSON file (default: None)
--config-schema, --no-config-schema Print configuration schema and exit (default: False)
--log-config Path Load logging configuration from YAML/JSON file - see <https://docs.python.org/3/library/logging.config.html#logging-config-dictschema> (default: None)
--cs-hub str (default: cshub)

silkwave options:

--silkwave.name str Description to identify this client on the SILKWAVE network (default: None)

silkwave.alias options:

default: None (undefined)

Alternative identity to register with a nameserver

--silkwave.alias.nameserver str
Nameserver to register alias on (ifdef: required)
--silkwave.alias.name str
An alternative name to register with the nameserver (ifdef: required)
--silkwave.alias.prefix-hub bool
Prefix identity with hub name e.g.
{nameserver}:{hub}:{name} (default: False)

silkwave.security options:

The security marking to use on the envelope of all messages sent to the SILKWAVE hub where a more specific marking is not supplied by the sender.

--silkwave.security.classification {U,R,C,S,TS}
(default: U)
--silkwave.security.sci-controls str
(default: None)
--silkwave.security.releaseable-to str
(default: None)
--silkwave.security.access-controls str
(default: None)
--silkwave.security.owner-producer str
(default: None)
--silkwave.security.creator-organization str
(default: None)
--silkwave.security.classified-by str
(default: None)
--silkwave.security.classification-reason str
(default: None)
--silkwave.security.derived-from str
(default: None)
--silkwave.security.declass-date str
(default: None)
--silkwave.security.declass-exception str
(default: None)
--silkwave.security.auth-id str
(default: None)
--silkwave.security.alt-label str
(default: None)

silkwave.connection options:

Configuration for the SILKWAVE connection

--silkwave.connection.host str
Hostname of the SILKWAVE hub to connect to (default: localhost)
--silkwave.connection.port int

```

        Port number of the SILKWAVE hub to connect to.
        (default: 61623)
--silkwave.connection.username str
    Username to authenticate with SILKWAVE hub (default:
    client)
--silkwave.connection.password str
    Password to authenticate with SILKWAVE hub (default:
    manager)

silkwave.connection.ssl options:
    SSL configuration for SILKWAVE connections
    Set to 'false' to turn off SSL
    Set to 'true' to use a default SSL configuration

--silkwave.connection.ssl.key Path
    PEM file containing SSL private key (default: None)
--silkwave.connection.ssl.passphrase str
    Passphrase to decrypt SSL private key (default: None)
--silkwave.connection.ssl.certificate Path
    PEM file containing SSL certificate, or certificate
    and key (default: None)
--silkwave.connection.ssl.ca Path
    PEM file containing trusted CA certificates (default:
    None)
--silkwave.connection.ssl.ca-dir Path
    Path to OpenSSL truststore directory containing CA
    certificates (default: None)

silkwave.files options:
    Configuration for file transfers

--silkwave.files.force-hub-hostname str
    Override hostname of the local SILKWAVE hub (default:
    None)
--silkwave.files.force-hub-port int
    Override port of the local SILKWAVE hub (default:
    None)

silkwave.files.https options:
    SSL configuration for HTTPS connections
    Set to 'false' to use HTTP
    Set to 'true' to use same settings as SILKWAVE connection

--silkwave.files.https.key Path
    PEM file containing SSL private key (default: None)
--silkwave.files.https.passphrase str
    Passphrase to decrypt SSL private key (default: None)
--silkwave.files.https.certificate Path
    PEM file containing SSL certificate, or certificate
    and key (default: None)
--silkwave.files.https.ca Path
    PEM file containing trusted CA certificates (default:
    None)
--silkwave.files.https.ca-dir Path
    Path to OpenSSL truststore directory containing CA
    certificates (default: None)

silkwave.streams options:
    Configuration for streams

--silkwave.streams.force-hub-hostname str
    Override hostname of the local SILKWAVE hub (default:
    None)
--silkwave.streams.listeners list[JSON]
    UDP listeners (default: [])

```

```

silkwave.streams.listener-default options:
  Default configuration for stream listeners

--silkwave.streams.listener-default.host str
    Hostname for clients to send to (default: Tristans-
    MacBook-Pro)
--silkwave.streams.listener-default.bind str
    Interface to bind to (default: 0.0.0.0)

client options:
--client.registrant-id str
    Registrant id of this client (default:
    urn:csmf:pys:cs-
    client-89189dc1-3d13-49f6-b30a-5ad7a3681522)
--client.force-span-context bool
    Force a SpanContextMf onto all outgoing messages even
    when not in message definition (default: True)

client.security options:
  Default security marking for outgoing messages

--client.security.classification {U,R,C,S,TS}
    (default: U)
--client.security.sci-controls str
    (default: None)
--client.security.releaseable-to str
    (default: None)
--client.security.access-controls str
    (default: None)
--client.security.owner-producer str
    (default: None)
--client.security.creator-organization str
    (default: None)
--client.security.classified-by str
    (default: None)
--client.security.classification-reason str
    (default: None)
--client.security.derived-from str
    (default: None)
--client.security.declass-date str
    (default: None)
--client.security.declass-exception str
    (default: None)
--client.security.auth-id str
    (default: None)
--client.security.alt-label str
    (default: None)

client.signing options:
  Message signing configuration

--client.signing.key Path
    PEM file containing signing private key (default:
    None)
--client.signing.passphrase str
    Passphrase to decrypt signing private key (default:
    None)
--client.signing.certificate Path
    PEM file containing signing certificate, or
    certificate and key (default: None)
--client.signing.ca Path
    PEM file containing trusted CA certificates for
    signature verification (default: None)
--client.signing.allowed-x509-subjects list[str]
    List of X509 subjects to accept messages from
    (default: [])

```

```

--client.signing.registration-certificate Path
    PEM file containing SSL certificate of the trusted
    registration service (default: None)
--client.signing.check-signatures, --no-client.signing.check-signatures
    Check signatures on incoming messages (default: False)
--client.signing.allow-unsigned, --no-client.signing.allow-unsigned
    Allow unsigned messages to be processed (default:
    True)

client.validation options:
    Message validation configuration

--client.validation.incoming bool
    Validate incoming messages (default: False)
--client.validation.outgoing bool
    Validate outgoing messages (default: False)

otel.service options:
--otel.service.name str
    service.name for OTEL Resource (default: PyCS Client)
--otel.service.namespace str
    service.namespace for OTEL Resource (default: Common
    Services)

otel.connection options:
--otel.connection.host str
    Host name of metrics collector (default: localhost)
--otel.connection.port int
    Port number of metrics collector (default: 4317)

otel.connection.ssl options:
    SSL configuration for gRPC connections
    Set to 'false' to turn off SSL
    Set to 'true' to use a default SSL configuration

--otel.connection.ssl.key Path
    PEM file containing SSL private key (default: None)
--otel.connection.ssl.passphrase str
    Passphrase to decrypt SSL private key (default: None)
--otel.connection.ssl.certificate Path
    PEM file containing SSL certificate, or certificate
    and key (default: None)
--otel.connection.ssl.ca Path
    PEM file containing trusted CA certificates (default:
    None)
--otel.connection.ssl.ca-dir Path
    Path to OpenSSL truststore directory containing CA
    certificates (default: None)

otel.tracer options:
--otel.tracer.max-queue-size int
    (default: None)
--otel.tracer.max-export-batch-size int
    (default: None)
--otel.tracer.schedule-delay-millis float
    (default: None)
--otel.tracer.export-timeout-millis float
    (default: None)

otel.metrics options:
--otel.metrics.export-interval-millis float
    (default: 1000)
--otel.metrics.export-timeout-millis float
    (default: None)

otel.log options:

```

```
--otel.log.max-queue-size int
    (default: None)
--otel.log.max-export-batch-size int
    (default: None)
--otel.log.schedule-delay-millis float
    (default: None)
--otel.log.export-timeout-millis float
    (default: None)
--otel.log.level {CRITICAL,ERROR,WARNING,INFO,DEBUG}
    Minimum log level to send to OTEL (default:
    LogLevel.INFO)
```

8. OpenTelemetry Instrumentation

The `pys.client` library is instrumented with OpenTelemetry (OTEL) Logs, Metrics and Traces. == Configuration

An `OTELConfiguration` class is provided to set up appropriate default OTEL tracers, meters and loggers. Populating an instance of this class and calling `.configure()` will set up OTLP exporters for metrics, traces and logs, and instrument any `asyncio` calls.

The configuration class is a Pydantic data model, and as such can be included within an `ApplicationSettings` class, as detailed in [Section 7, “Configuration”](#).

Use of this class is not mandatory, so if it is not suitable for a particular application, alternative configurations may be created as defined in the [OTEL documentation](#).

8.1. Metrics

8.1.1. Sending Messages

As each message is sent the `messaging.sent.messages` counter is incremented for each destination node found in the `MsgDistMf` microformat. The following attributes are populated from the `MsgDistMf` and `MsgMetadataMf` microformats.

attribute	source
<code>messaging.system</code>	"cs"
<code>messaging.csmf.message.type.id</code>	<code>MsgDistMf.type.id</code>
<code>messaging.csmf.message.type.version</code>	<code>MsgDistMf.type.version</code>
<code>messaging.csmf.message.dest.end_point</code>	<code>MsgDistMf.destination[i].endPoint</code>
<code>messaging.csmf.message.dest.registrant_id</code>	<code>MsgDistMf.destination[i].registrantId</code>
<code>messaging.csmf.message.dest.node_id</code>	<code>MsgDistMf.destination[i].nodeId</code> (if present)
<code>messaging.csmf.message.dest.kind</code>	<code>MsgMetadataMf.destinationKind</code>

8.1.2. Received Messages

As each message is received the `messaging.consumed.messages` counter is incremented. The following attributes are populated from the `MsgDistMf` and `MsgMetadataMf` microformats.

attribute	source
<code>messaging.system</code>	"cs"
<code>messaging.csmf.message.type.id</code>	<code>MsgDistMf.type.id</code>
<code>messaging.csmf.message.type.version</code>	<code>MsgDistMf.type.version</code>
<code>messaging.csmf.message.origin.end_point</code>	<code>MsgDistMf.originator.endPoint</code>
<code>messaging.csmf.message.origin.registrant_id</code>	<code>MsgDistMf.originator.registrantId</code>
<code>messaging.csmf.message.origin.node_id</code>	<code>MsgDistMf.originator.nodeId</code> (if present)

attribute	source
<code>messaging.csmf.message.origin.kind</code>	<code>MsgMetadataMf.originatorKind</code>

8.2. Traces

8.2.1. Sending Messages

A tracing span is created around each message sent. If the message to be sent has a `SpanContextMf` microformat in its definition and the caller has not populated this, it will be populated with the information regarding the current span context. This enable spans in receiving processes to be associated so that flows can be traced across processes. This microformat may be forced into the message even for those which do not have it defined by setting `ClientConfig.force_span_context` to `True` (the default) when creating the `CommonServicesClient`, thus enabling tracing across IDD's which have not yet been updated to include it.

By default, the span will be named "`Send {message_class_name} Message`" but this can be overridden by passing a `span_name` argument to the `send_message` call, or a `request_span_name` parameter to `send_request`, `request_responses`, `request_callback` or `request_callbacks`.

8.2.2. Receiving Messages

A tracing span is created around each message received. If a `SpanContextMf` microformat is contained in the received message, this message span is associated with the context defined in the microformat.

By default, the span will be named "`Process {message_class_name} Message`", but this can be changed by passing a `span_name` argument to `subscribe` or `subscribe_callback`, or a `response_span_name` argument to `send_request`, `request_responses`, `request_callback` or `request_callbacks`.

The context created by the subscribe or request call is kept open for as long as the context manager is open to allow child spans to be created by client code, and any log messages or events to be associated.

Deprecated `request_response` function.

The `request_response` functions on `CommonServicesClient` and `CommonServicesNode` have been deprecated in favour of the `send_request` context manager. When using `request_response` there is no way to keep the span context open and allow child spans, events or logs to be associated.

Deprecated code

```
request = await client.request_response(...)
...
```

Recommended code

```
async with client.send_request(...) as response:
```


8.2.3. Span Attributes

The following attributes are populated on both send and receive message spans from the message microformats.

attribute	source
<code>messaging.system</code>	"cs"
<code>messaging.message.id</code>	<code>SpanContextMf.message_id</code>
<code>messaging.message.conversation_id</code>	<code>MsgDistMf.requestId</code>
<code>messaging.operation.name</code>	<code>MsgDistMf.type.id</code>
<code>messaging.csmf.security.classification.level</code>	<code>message.security.classification</code>
<code>messaging.csmf.message.type.id</code>	<code>MsgDistMf.type.id</code>
<code>messaging.csmf.message.type.version</code>	<code>MsgDistMf.type.version</code>
<code>messaging.csmf.message.origin.end_point</code>	<code>MsgDistMf.originator.endPoint</code>
<code>messaging.csmf.message.origin.registrant_id</code>	<code>MsgDistMf.originator.registrantId</code>
<code>messaging.csmf.message.origin.node_id</code>	<code>MsgDistMf.originator.nodeId</code> (if present)
<code>messaging.csmf.message.origin.kind</code>	<code>MsgMetadataMf.originatorKind</code>
<code>messaging.csmf.message.dest.end_point</code>	<code>MsgDistMf.destination[0].endPoint</code>
<code>messaging.csmf.message.dest.registrant_id</code>	<code>MsgDistMf.destination[0].registrantId</code>
<code>messaging.csmf.message.dest.node_id</code>	<code>MsgDistMf.destination[0].nodeId</code> (if present)
<code>messaging.csmf.message.dest.kind</code>	<code>MsgMetadataMf.destinationKind</code>
<code>messaging.csmf.message.dest.{i}.end_point</code>	<code>MsgDistMf.destination[i].endPoint</code>
<code>messaging.csmf.message.dest.{i}.registrant_id</code>	<code>MsgDistMf.destination[i].registrantId</code>
<code>messaging.csmf.message.dest.{i}.node_id</code>	<code>MsgDistMf.destination[i].nodeId</code> (if present)
<code>messaging.csmf.message.dest.kind</code>	<code>MsgMetadataMf.destinationKind</code>
<code>messaging.csmf.message.extensions.{i}.id</code>	<code>MsgDistMf.extensions[i].id</code>
<code>messaging.csmf.message.extensions.{i}.version</code>	<code>MsgDistMf.extensions[i].version</code>
<code>messaging.csmf.message.super.{i}.id</code>	<code>MsgDistMf.super[i].id</code>
<code>messaging.csmf.message.super.{i}.version</code>	<code>MsgDistMf.super[i].version</code>
<code>messaging.csmf.message.content.kind</code>	<code>MsgMetadataMf.contentKind</code>
<code>messaging.csmf.message.outcome.result</code>	<code>OutcomeMf.outcome</code> (if present)
<code>messaging.csmf.message.outcome.desc</code>	<code>OutcomeMf.description</code> (if present)
<code>messaging.csmf.message.outcome.detail</code>	<code>OutcomeMf.detail</code> (if present)
<code>error.type</code>	<code>OutcomeMf.description</code> (if present)

8.3. Logging

All logs will be forwarded to OTEL given a suitable logging configuration, such as that provided by

the `OTELConfiguration` class.

8.4. Client Applications

Client applications should create their own tracer and meter and create spans and metrics to their own requirements. Any spans created within the context of a received message or response will be child of the message span created by the `pys.client` package. Any message spans for sent messages created by the `pys.client` package will be a child of the current span at the point of the call.

```
from opentelemetry import trace

config = OTELConfiguration(service=ServiceConfig(name="Tasker"))
config.configure()

tracer = trace.get_tracer("example.application")

class Tasker(CommonServicesNode):

    ...

    async def tune(
        self,
        message: Tunercommandrequest,
        destination: Registrant):

        with tracer.start_as_current_span("Tune Command"):
            async with self.send_request(
                message = message,
                destination = destination,
                expected_response=TunerCommandResponse,
            ) as response:
                tracer.start_as_current_span("Tune Response")
```

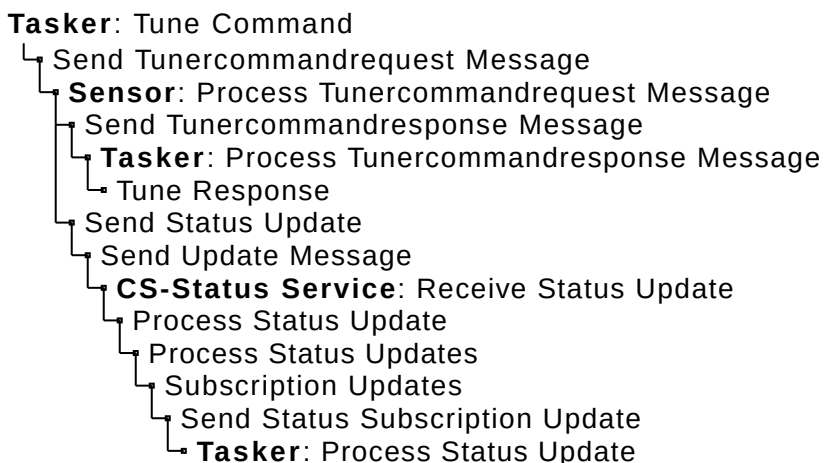


Figure 1. OTEL Trace